Practical 2: Mandelbrot Set Performance Benchmarking Using the MATLAB Parallel Computing Toolbox

Samson Okuthe
Department of Electrical Engineering
University of Cape Town
OKTSAM001

Nyakallo Peete
Department of Electrical Engineering
University of Cape Town
PTXNYA001

***Abstract*—This report investigates the efficacy of multicore parallelism by benchmarking the generation of the Mandelbrot set. Using MATLAB's Parallel Computing Toolbox (PCT), a sequential nested-loop algorithm was adapted into a parallel `parfor` architecture. Execution times were evaluated across eight ascending resolutions, from 0.48 Megapixels (SVGA) to 33.18 Megapixels (8K UHD). Furthermore, core utilization was analyzed by scaling the parallel worker pool up to 10 physical CPU cores. Results demonstrate that while small workloads suffer from severe inter-thread communication overhead, massive parallel workloads scale highly efficiently, achieving a peak speedup of 4.55x on 10 cores for the 5K resolution. The study highlights vital parallel computing principles, notably Amdahl's Law, thread overhead, and dynamic load balancing.**

***Index Terms*—Mandelbrot Set, Parallel Computing Toolbox, parfor, Amdahl's Law, Load Balancing**

I. Introduction

As computational demands in embedded systems and scientific computing scale beyond the limitations of single-core clock speeds, leveraging multicore processor architectures becomes essential. The MATLAB Parallel Computing Toolbox (PCT) provides a high-level framework for distributing computational tasks across multiple threads using worker pools.

The generation of the Mandelbrot set serves as an ideal baseline for benchmarking parallel efficiency. The fractal is generated by iterating the complex sequence $f_c(z) = z^2 + c$ starting from $z = 0$, tracking how many iterations are required for the magnitude to exceed an escape condition ($|z| \leq 2$). Because the escape calculation for any given pixel coordinate (x, y) requires no information from neighboring pixels, the problem possesses no data dependencies. This defines it as an "embarrassingly parallel" workload, allowing us to evaluate raw parallel speedup, overhead penalties, and the limits of Amdahl's Law without the hindrance of complex inter-thread communication.

II. Methodology

A. Algorithmic Implementation

Two computational methods were developed to map the complex coordinate region ($x \in [-2.0, 0.5], y \in [-1.2, 1.2]$) mapped to a maximum of 1000 iterations:

- **Sequential (`mandelbrot_sequential`):** Implemented using a double nested `for`-loop, evaluating the pixels systematically row-by-row on a single thread.
- **Parallel (`mandelbrot_parallel`):** The outer loop iterating over the image rows was replaced with a PCT `parfor` construct. To prevent memory access violations and excessive broadcast overhead, the image matrix was structured as a *sliced variable*. Each worker thread computed an entire 1D row array independently before returning it to the master thread for matrix assembly.

B. Benchmarking Strategy

Testing was executed across eight resolutions spanning from SVGA to 8K UHD. To isolate computational time from MATLAB's initialization latency, the 10-worker parallel pool (`gcp`) was forcibly started before any timing began. For the parallel tests, the worker count was incrementally adjusted across 2, 5, and 10 workers to evaluate scaling efficiency. Execution time was measured using high-resolution `tic`/`toc` timers.

III. Results

The visual outputs of both algorithms were verified to be identical, confirming successful parallel execution without race conditions. A summary of the execution times comparing the sequential baseline to the maximum 10-core parallel pool is presented in Table I.

Figure 1 illustrates the scaling of execution time relative to the image pixel density, while Figure 2 graphs the resulting speedup factor against the number of active workers for the 8K UHD image.

IV. Discussion

The benchmarking results exhibit three primary phenomena inherent to multi-core programming:

1. Thread Overhead on Small Workloads: On the smallest resolutions (SVGA and HD), the sequential execution actually outperformed the 10-worker parallel implementation. For SVGA, the serial code ran in 0.82 seconds, while the parallel pool took 2.29 seconds (a speedup of just 0.36x).

TABLE I
MANDELBROT EXECUTION TIMES AND SPEEDUP

Resolution	Megapixels	Serial (s)	Parallel 10w (s)	Speedup
SVGA	0.48	0.823	2.289	0.36x
HD	0.92	1.020	1.237	0.83x
Full HD	2.07	1.606	0.570	2.82x
2K	2.21	1.965	0.588	3.34x
QHD	3.69	2.710	0.826	3.28x
4K UHD	8.29	6.260	3.715	1.69x
5K	14.75	21.978	4.827	4.55x
8K UHD	33.18	48.278	12.510	3.86x

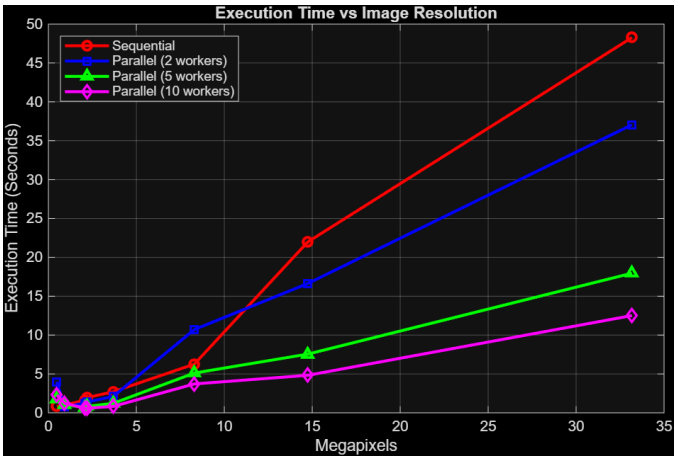

Fig. 1. Execution Time vs Image Resolution for varying worker counts.

The process of dispatching the row indices to 10 background workers, managing the memory space, and gathering the sliced arrays back to the master thread incurs a fixed latency penalty. Because the raw sequential computation time for SVGA is so brief, this inter-process communication overhead dominates, severely degrading parallel efficiency.

2. Dynamic Load Balancing in `parfor`: The Mandelbrot set is heavily asymmetric in its computational cost. Pixels within the fractal boundary reach the full 1000 iterations, whereas pixels in the outer bounds escape within a few iterations. If the workload had been statically distributed, threads processing the dense regions would bottleneck the system while others sat idle. MATLAB's `parfor` successfully circumvents this by utilizing dynamic load balancing, automatically assigning new rows to workers as they become available. This guarantees optimal utilization of all 10 cores as the image scales.

3. Amdahl's Law and Scalability Limits: As the workload scales up to 5K and 8K UHD, the compute time vastly outweighs the worker overhead, resulting in highly efficient parallel scaling. However, as shown in Figure 2, the actual speedup curve significantly diverges from the ideal linear scaling line. At 10 workers on the 8K image, the system achieved a 3.86x speedup, rather than a 10x speedup. According to Amdahl's Law, maximum speedup is bottlenecked by the strictly serial portions of the code. In this implementation, the serial initialization of the matrices, the final memory gathering of the data chunks, and memory bus saturation prevent true 100% efficiency, causing performance gains to plateau as more cores are added.

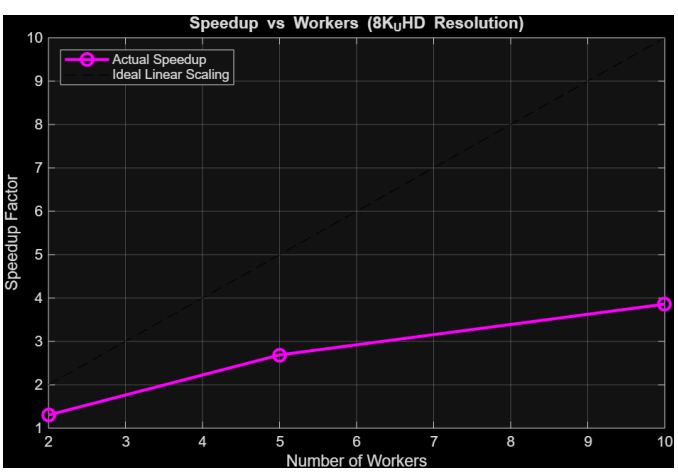

Fig. 2. Speedup Factor vs Active Workers demonstrating Amdahl's Law.

V. CONCLUSION

This practical validated the capabilities of the MATLAB Parallel Computing Toolbox. Translating a nested serial loop into a `parfor` architecture successfully reduced computation time for generating the 33.18 Megapixel 8K Mandelbrot set from 48.28 seconds down to 12.51 seconds (a 3.86x speedup on 10 cores). The data confirms that while "embarrassingly parallel" workloads yield massive performance gains on large datasets, small workloads suffer heavily from thread dispatch overhead. Furthermore, true linear speedup is impossible due to memory assembly limitations dictated by Amdahl's Law. For future improvements on massive datasets, utilizing `gpuArray` to offload the coordinate generation to a GPU would yield far superior execution times by leveraging thousands of lightweight SIMD cores rather than heavy CPU threads.